

P Prode Carestino

Aplicacion de predicciones para el Mundial de Futbol 2026

GUIA COMPLETA DE ESTRUCTURA, COMPONENTES E INTERACCION

Este documento ha sido disenado especificamente para comprender de manera intuitiva y profunda como funciona una aplicacion web full-stack, analizando detalladamente cada carpeta, subcarpeta y archivo dentro del proyecto 'prode-carestino'.

Si programas con Inteligencia Artificial, esta guia te ayudara a entender como los diferentes componentes se comunican entre si (Frontend, Backend, Base de Datos y Motor de Reglas) para formar un sistema completo y escalable.

DESTINATARIO:

Francisco Fernandez

TECNOLOGIAS CLAVE:

Fastify (API Node.js) | React + Vite (Frontend) | Prisma ORM | SQLite DB | Tailwind CSS

FECHA DE COMPILACION:

Mayo de 2026

1. Introduccion y Arquitectura General

El proyecto 'prode-carestino' es una aplicacion web full-stack disenada para gestionar un Prode (juego de predicciones deportivas) interno durante el Mundial de Futbol de 2026. Esta dimensionado para un grupo de aproximadamente 100 usuarios activos y dura un mes.

Como interactuan las piezas de la aplicacion?

Toda aplicacion web moderna consta de tres capas principales que colaboran entre si. Aqui te explicamos el ciclo de vida de una interaccion (por ejemplo, cuando un usuario guarda un pronostico para un partido):

- **1. La Capa de Presentacion (Frontend):** Es la interfaz visual con la que interactua el usuario en su navegador. Esta construida en React con Vite. Cuando el usuario escribe el marcador de un partido (ej. Argentina 2 - Brasil 1) y hace clic en 'Guardar', el frontend captura estos datos y envia una peticion HTTP (POST/PUT) con formato JSON hacia el backend.
- **2. La Capa de Logica de Negocio (Backend):** Es el servidor de aplicacion, construido con Fastify (un framework de Node.js ultra-rapido). El servidor recibe la peticion HTTP. Primero, valida que los datos sean correctos (usando esquemas de Zod, como confirmar que los goles no sean numeros negativos). Luego, ejecuta las reglas (por ejemplo, verificar si el partido aun no ha comenzado y si la prediccion es legal). Finalmente, invoca a Prisma para interactuar con la base de datos.
- **3. La Capa de Datos (Base de Datos):** Es donde la informacion se guarda de forma persistente. Usamos SQLite (un archivo ligero llamado `prode.db`) gestionado por Prisma ORM. Prisma traduce las llamadas de codigo de JavaScript/TypeScript a sentencias SQL para insertar o actualizar los datos de la prediccion de forma segura.
- **4. El Motor de Reglas y Puntuacion:** Cuando el administrador (admin) de la aplicacion ingresa un resultado real de un partido, se activa un proceso de puntuacion (scoring). Este lee todas las predicciones de ese partido guardadas en la base de datos, calcula los puntos de cada usuario segun las reglas oficiales, y actualiza los puntajes individuales. Automaticamente se actualiza la tabla de posiciones (leaderboard).

2. Arbol General del Directorio

A continuacion, se muestra la estructura visual de las carpetas y archivos mas importantes del proyecto. Esta jerarquia ayuda a ubicar donde esta cada elemento del frontend y backend:

JERARQUIA DEL PROYECTO (DIRECTORIO COMPLETO)

```
prode-carestino/
|-- backend/                                # Servidor API de Node.js + Prisma ORM
|   |-- prisma/                             # Carpeta de Base de Datos (SQLite + Modelos)
|   |   |-- migrations/                     # Historial de cambios en las tablas
|   |   |-- schema.prisma                   # Definicion de tablas de datos y relaciones
|   |   \-- seed.ts                         # Script para llenar datos semilla iniciales
|   |-- src/                                #Codigo fuente de la API backend
|   |   |-- index.ts                        # Punto de entrada del servidor Fastify
|   |   |-- lib/                           # Librerias y configuraciones globales compartidas
|   |   |-- plugins/                       # Modulos de extension para Fastify (Autenticacion, DB)
|   |   \-- modules/                       # Funcionalidades especificas por dominio (Modulos)
|   |       |-- admin/                     # Panel de control de administracion
|   |       |-- auth/                      # Login, intentos fallidos y bloqueos
|   |       |-- predictions/               # Guardado y validacion de pronosticos
|   |       |-- scoring/                   # Motor de asignacion de puntos
|   |       \-- tournament/               # Datos de partidos y tabla de posiciones
|   |-- package.json                       # Dependencias del backend (librerias de Node)
|   \-- tsconfig.json                      # Configurar compilador TypeScript
|-- frontend/                              # Cliente Web (React SPA + Tailwind CSS)
|   |-- src/                               #Codigo fuente de la interfaz visual
|   |   |-- App.tsx                        # Componente principal de React
|   |   |-- index.css                      # Estilos globales y tokens visuales de CSS
|   |   \-- main.tsx                       # Punto de arranque del render de React
|   |-- package.json                       # Dependencias del frontend (React, Tailwind)
|   |-- tailwind.config.js                 # Configuracion de tokens de disenio visual
|   \-- vite.config.ts                     # Configurar el empaquetador de desarrollo Vite
\-- docs/                                  # Documentacion de Mockups y disenios UI
    \-- prode_mockup.jsx                   # Mockup inicial del flujo interactivo
```

3. Guia Detallada de Carpetas y Subcarpetas

Cada carpeta del proyecto tiene una responsabilidad clara y unica. Mantener esta division es lo que permite escalar y entender la aplicacion facilmente, especialmente trabajando con IA.

Raiz del Proyecto

- **backend:** Contiene el backend (API) de la aplicacion. Acepta solicitudes del navegador, valida permisos, actualiza la base de datos y contiene las reglas del juego.
- **frontend:** Contiene el codigo visual e interactivo de la interfaz web que los usuarios ven y tocan en el navegador.
- **docs:** Guarda documentacion de arquitectura o maquetacion inicial del disenio interactivo, como ``prode_mockup.jsx``.

Subcarpetas de backend/

- **backend/prisma:** Contiene todo lo relacionado con la base de datos. Los archivos que definen la estructura (schema.prisma), los scripts para inicializar la base de datos con equipos y partidos (seed), y las migraciones que aplican los cambios estructurales.
- **backend/src:** Codigo fuente central de Node.js + TypeScript del servidor.
- **backend/src/lib:** Librerias comunes del backend: archivo de conexion unica a la base de datos, validador de variables de entorno y definicion de errores globales.
- **backend/src/plugins:** Plugins de Fastify. En Fastify, los plugins inyectan funcionalidades transversales. Aqui configuramos las cookies de sesion, el mecanismo de login JWT y la inyeccion de base de datos.
- **backend/src/modules:** El core del backend. Organizado bajo el patron de arquitectura limpia o modular. Cada carpeta adentro resuelve un dominio o funcionalidad completa (esquemas, rutas y logica de base de datos unidos).

Modulos del Backend (backend/src/modules/)

- **modules/auth:** Gestiona el registro de usuarios, el login, la generacion del token y las politicas de seguridad (como bloquear un usuario despues de varios intentos fallidos).
- **modules/admin:** Operaciones restringidas solo a administradores, como ingresar resultados de partidos de futbol reales o gestionar y desactivar cuentas de usuarios.
- **modules/predictions:** Gestiona el registro de pronosticos que hace cada empleado para los partidos, validando que no se guarde despues del pitazo inicial.
- **modules/scoring:** Implementa la logica pura de puntajes. Evalua la prediccion vs el resultado real y retorna los puntos ganados.
- **modules/tournament:** Consulta de grupos de la Copa del Mundo, informacion de partidos de cada ronda y compilacion del leaderboard (tabla de posiciones del juego organizada por mayor puntaje acumulado).

Subcarpetas de frontend/

- **frontend/src**: Contiene el código React que se compila y carga en el navegador.

4. Archivos Clave y su Funcionamiento

A continuacion, listamos los archivos clave y que funcion exacta cumple cada uno. Esta seccion te permitira navegar entre el codigo fuente sabiendo que buscar en cada archivo.

Archivos de Base de Datos (backend/prisma/)

- **schema.prisma:** Define el modelo de datos. Contiene tablas como:
 - User: Almacena email, cuil, nombre, hash de password y rol.
 - Match: Almacena el partido (equipos, fecha, goles reales, estado).
 - Prediction: Guarda los goles pronosticados por un usuario, los puntos obtenidos y si fue exacto.
 - LoginAttempt: Controla los intentos fallidos de login para bloquear ataques.
 - TournamentRound: Define las fechas o fases (Fase de Grupos, Octavos, etc.).
- **seed.ts / seed-dev.ts:** Permiten cargar informacion inicial en la base de datos limpia. Si ejecutas ``npm run seed``, este archivo inserta automaticamente los paises del Mundial 2026, los grupos A al L y los partidos basicos para que la aplicacion funcione de entrada sin tener que tipear todo a mano.

Archivos de Entrada y Configuracion Backend (backend/src/)

- **index.ts:** Es el cerebro del backend. Inicia el servidor HTTP, monta el manejador de errores global para traducir fallas inesperadas a JSON legible (como capturar errores de Zod o de Base de Datos) y registra todas las rutas de la API (bajo el prefijo `/api`).
- **lib/db.ts:** Crea e inicializa una unica instancia de PrismaClient. De esta forma, todas las consultas a la base de datos desde cualquier parte del servidor pasan por la misma conexion, optimizando el rendimiento.
- **lib/env.ts:** Carga el archivo `.env` y valida las variables con Zod. Si falta una variable critica (como `JWT_SECRET` o `DATABASE_URL`) en el entorno, el servidor se detiene de inmediato indicando exactamente que falta.
- **lib/errors.ts:** Definicion de las excepciones personalizadas del sistema (como `BadRequestError`, `AuthError`, `NotFoundError`). El manejador en `index.ts` lee estas clases y sabe exactamente que codigo HTTP retornar (400, 401, 404, etc.).

Plugins del Backend (backend/src/plugins/)

- **plugins/auth.ts:** Verifica si un usuario inicio sesion de forma valida mediante cookies seguras que guardan un token JWT firmado. Inyecta un ayudante llamado ``request.jwtVerify()`` para proteger las rutas privadas.
- **plugins/cookies.ts:** Configura la recepcion y envio de cookies en Fastify, definiendo politicas de cookies HTTP-Only y seguras para evitar que scripts maliciosos roben la sesion.
- **plugins/db.ts:** Registra a Prisma de forma global como plugin de Fastify, haciendo posible acceder a la base de datos como ``app.prisma`` en lugar de importar la base de datos en cada controlador.

5. Logica de los Modulos y Motor de Reglas

Los modulos contienen una arquitectura interna repetitiva orientada a servicios. Se componen de:

1. ``routes.ts``: Declara las rutas URL de la API y que controlador las maneja.
2. ``schemas.ts``: Estructura del formato de entrada/salida validado con la libreria Zod.
3. ``service.ts``: Escribe las consultas Prisma y los algoritmos que modifican los datos.

Modulo de Puntuacion (backend/src/modules/scoring/scoring.ts)

Este es un archivo sumamente especial porque implementa la logica de reglas pura (sin base de datos). Contiene el algoritmo ``calculatePoints`` que recibe la prediccion del usuario y el resultado real:

- **Puntaje Exacto (7 puntos)**: Si el usuario adivino los goles exactos de ambos equipos (ej: apostó 2-1 y terminó 2-1).
- **Ganador + Diferencia (5 puntos)**: Si adivino quien ganaba (o si empataron) y además la diferencia de goles exacta, pero no el marcador final (ej: apostó 3-1 y terminó 2-0. En ambos la diferencia es de 2 goles y gana el local).
- **Solo Ganador o Empate comun (3 puntos)**: Si adivino solo que ganaba el equipo local/visitante, o que empatarian, pero sin dar con la diferencia o goles (ej: apostó 1-0 y terminó 3-1. Gana el local en ambos casos, pero con diferente diferencia).
- **Caso Especial de Penales (3 puntos)**: En partidos de eliminación directa (knockout) que terminen empatados tras 120 minutos, el resultado oficial para el prode es Empate. Pero la regla premia con 3 puntos adicionales a quien adivino que equipo ganaba en la tanda de penales.

6. Estructura y Configuración del Frontend

El frontend de la aplicación se ejecuta directamente en el navegador del usuario y está diseñado para ser veloz y visualmente amigable.

- **frontend/src/main.tsx:** Es el punto de arranque. Importa la librería React y monta el componente ``App.tsx`` dentro del elemento `div` con id `'root'` en el archivo ``index.html``.
- **frontend/src/App.tsx:** Componente principal de React. Contiene la interfaz general del sitio. Actualmente sirve como un componente de prueba para verificar que la tipografía, los estilos visuales y la configuración de Tailwind funcionen correctamente. Más adelante se colocarán aquí las rutas de las pantallas principales (Login, Pronósticos, Leaderboard, etc.).
- **frontend/src/index.css:** Define los estilos globales del proyecto. Configura Tailwind CSS y establece las variables de la paleta de colores oficial (como el naranja corporativo de Carestino, tonos de superficie y fuentes).

Archivos de Configuración Root y Compilación

- **tailwind.config.js (Frontend):** Define los colores de la marca, los tamaños de fuentes y espaciados personalizados de CSS para que se puedan aplicar en toda la aplicación usando clases simples.
- **vite.config.ts (Frontend):** Configuración del servidor de desarrollo Vite. Permite arrancar el sitio en milisegundos en modo de desarrollo y compila los archivos JS minimizados de forma automática para producción.
- **package.json (Backend & Frontend):** Es el inventario del proyecto. Enlista todas las librerías externas que usa la aplicación (como Fastify, React, Tailwind, Prisma, Zod) y define los comandos básicos para ejecutar la aplicación (`npm run dev`).

7. Guia para Programar este Proyecto con IA

Como programador que delega la creacion de codigo a la Inteligencia Artificial (IA), tu rol cambia de 'escribir lineas de codigo' a 'ser un arquitecto que valida y dirige'. Aqui tienes 3 consejos basicos para este proyecto:

- **1. Haz cambios incrementales:** No le pidas a la IA que construya todo el frontend o backend en una sola solicitud. Pide cosas especificas, como 'Agrega un boton en App.tsx para navegar a predicciones' o 'Modifica predictions.service.ts para validar la hora limite de envio de pronosticos'. De esta forma, sera mas facil detectar si la IA comete un error.
- **2. Usa los Esquemas como tu contrato de datos:** En este proyecto, los archivos ``.schemas.ts`` de Zod son la ley de lo que entra y sale de la API. Si la IA modifica el frontend y genera errores de validacion (HTTP 400), revisa el schema correspondiente para ver que campo falta enviar o si el tipo es incorrecto (ej: mandar un String cuando el backend espera un Entero).
- **3. Aprovecha Prisma para debuggear la Base de Datos:** Puedes abrir una vista interactiva de tu base de datos ejecutando ``npx prisma studio`` en la carpeta ``backend``. Esto levantara una interfaz web donde podras ver los usuarios, editar partidos de prueba y confirmar que las predicciones esten asignando los puntos correctos segun el motor de reglas sin tener que escribir consultas SQL.